

```
In [1]: import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
import matplotlib.pyplot as plt
```

```
In [2]: df=pd.read_csv("Social_Network_Ads.csv")
```

```
In [3]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 400 entries, 0 to 399
Data columns (total 5 columns):
#   Column                Non-Null Count  Dtype
---  -
0   User ID               400 non-null   int64
1   Gender                400 non-null   object
2   Age                   400 non-null   int64
3   EstimatedSalary       400 non-null   int64
4   Purchased             400 non-null   int64
dtypes: int64(4), object(1)
memory usage: 15.8+ KB
```

```
In [4]: df.isnull().sum()
```

```
Out[4]: User ID          0
Gender              0
Age                0
EstimatedSalary    0
Purchased          0
dtype: int64
```

```
In [5]: df.describe()
```

```
Out[5]:
```

	User ID	Age	EstimatedSalary	Purchased
count	4.000000e+02	400.000000	400.000000	400.000000
mean	1.569154e+07	37.655000	69742.500000	0.357500
std	7.165832e+04	10.482877	34096.960282	0.479864
min	1.556669e+07	18.000000	15000.000000	0.000000
25%	1.562676e+07	29.750000	43000.000000	0.000000
50%	1.569434e+07	37.000000	70000.000000	0.000000
75%	1.575036e+07	46.000000	88000.000000	1.000000
max	1.581524e+07	60.000000	150000.000000	1.000000

```
In [6]: from sklearn.preprocessing import LabelEncoder
le=LabelEncoder()
```

```
df['gender_encoded']=le.fit_transform(df['Gender'])
df
```

Out[6]:

	User ID	Gender	Age	EstimatedSalary	Purchased	gender_encoded
0	15624510	Male	19	19000	0	1
1	15810944	Male	35	20000	0	1
2	15668575	Female	26	43000	0	0
3	15603246	Female	27	57000	0	0
4	15804002	Male	19	76000	0	1
...	...	...	...	...	...	...
395	15691863	Female	46	41000	1	0
396	15706071	Male	51	23000	1	1
397	15654296	Female	50	20000	1	0
398	15755018	Male	36	33000	0	1
399	15594041	Female	49	36000	1	0

400 rows × 6 columns

```
In [7]: #independent variable
x=df.drop("Gender",axis=1)
#x2=df.drop("User ID",axis=1)

#dependent variable
y=df["Purchased"]
```

```
In [8]: print(x.head())
```

	User ID	Age	EstimatedSalary	Purchased	gender_encoded
0	15624510	19	19000	0	1
1	15810944	35	20000	0	1
2	15668575	26	43000	0	0
3	15603246	27	57000	0	0
4	15804002	19	76000	0	1

```
In [9]: print(y.head())
```

```
0    0
1    0
2    0
3    0
4    0
Name: Purchased, dtype: int64
```

```
In [10]: x_train, x_test, y_train, y_test = train_test_split(x,y, test_size=0.2)
model=LogisticRegression()
```

```
model.fit(x_train,y_train)
y_pred=model.predict(x_test)
```

C:\Users\403\anaconda3\Lib\site-packages\sklearn\linear_model_logistic.py:473: ConvergenceWarning: lbfgs failed to converge after 100 iteration(s) (status=1):
STOP: TOTAL NO. OF ITERATIONS REACHED LIMIT

Increase the number of iterations to improve the convergence (max_iter=100).

You might also want to scale the data as shown in:

<https://scikit-learn.org/stable/modules/preprocessing.html>

Please also refer to the documentation for alternative solver options:

https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression

n_iter_i = _check_optimize_result(

```
In [11]: print("Predictions:", y_pred)
```

```
Predictions: [0 0 1 0 0 0 1 0 0 0 0 1 0 1 1 0 1 0 1 1 1 0 0 0 0 1 1 1 0 1 0 1 0 1 0
1 1
0 0 0 0 1 1 0 0 0 1 1 0 1 1 1 1 0 0 0 1 1 1 0 0 1 0 1 1 1 0 1 0 0 0 1 1 1
0 0 0 0 1 0]
```

```
In [12]: from sklearn.metrics import confusion_matrix
```

```
In [13]: cm= confusion_matrix(y_test,y_pred)
cm
```

```
Out[13]: array([[41,  6],
[ 2, 31]])
```

```
In [14]: print("TP= ",cm[1][1])
print("FP= ",cm[0][1])
print("TN= ",cm[0][0])
print("FN= ",cm[1][0])
```

```
TP= 31
FP= 6
TN= 41
FN= 2
```

```
In [15]: from sklearn.metrics import precision_score, recall_score,accuracy_score,f1_score
```

```
In [16]: precision= precision_score(y_test, y_pred)
print("precision= ",precision)
```

```
precision= 0.8378378378378378
```

```
In [17]: Accuracy= accuracy_score(y_test, y_pred)
print("accuracy= ",Accuracy)
```

```
accuracy= 0.9
```

```
In [18]: F1score= f1_score(y_test, y_pred)
print("F1score= ",F1score)
```

```
F1score= 0.8857142857142857
```

```
In [19]: Recallscore= recall_score(y_test, y_pred)
```

```
print("recall= ",Recallscore)
```

```
recall= 0.9393939393939394
```

```
In [ ]:
```